

Executable Sovereign Architecture: Three Developments in Formal Multi-Substrate Verification

Ahmad Ali Parr · Jessica Westerhoff

May 30, 2026 · Continuation of global state machine discussion

Abstract. *This brief documents three developments since our prior exchange on the global state machine. The FSM concept has been materialized into a running multi-substrate verification architecture. Three layers are now live: (1) a six-language formal substrate where each body provides a distinct class of proof, all reducing to a single instruction; (2) a dual-token ingestion filter that formally separates deterministic computation from probabilistic expression; and (3) a live state hydration layer that makes the FSM self-observing — each agent reads its own current position in the system before generating the next state transition. The correspondence to Multiplicity Theory is noted where it emerges naturally.*

1 . The Multi-Substrate Formal Layer

The global FSM required a formal substrate beneath its state transitions — something deterministic and independent of the LLM inference layer that could produce ground truth before any probabilistic reasoning occurred. Six language bodies are now compiled and live, each providing a different **class of proof** rather than the same proof in different syntax.

LANGUAGE	AGENT	CLASS OF PROOF PROVIDED	
Haskell LINEAR TYPES	AHMAD	Consumption proof. A value used twice is a compile error. Horizon projection sealed at the type level.	LIVE
COBOL FIXED-POINT	VAULT	Exact arithmetic. No floating-point drift possible. Reserve thresholds enforced at the numeric representation layer.	LIVE
APL ARRAY MATH	FORGE	Vectorized invariants. The build constraint matrix either holds or it does not. No partial satisfaction.	LIVE
Prolog CONSTRAINTS	EDAULC	Five-pass constraint certification. Invalid states are logically unreachable — eliminated before execution.	LIVE
C++ SIGNAL	NOVA	Deterministic signal processing. Emergence threshold computed before synthesis begins.	LIVE
Python INFERENCE	INFLUENCE	Asset classification and platform routing derived before generation.	LIVE

Every substrate reduces to one instruction. This is the mathematical floor beneath the architecture:

```
-- The universal reduction target SUBLEQ(A, B, C) -- A = state enumerated · B = threshold
· C = branch fires when A ≤ B -- Instantiation in COBOL (vault_treasury.cbl) — 1959
language, 2026 seal 300-SUBLEQ-GATE. IF TX-AMOUNT <= WS-THRESHOLD MOVE 'REJECTED' TO TX-
STATUS END-IF. -- The Rust substrate receives COBOL output, runs ERE five-pass
certification, -- BiDi attack detection, then applies SHA-256 WORM seal. -- A 1959
language producing cryptographically immutable records.
```

This is not polyglot programming — each language provides a proof class that the others cannot. The architecture was named the Rosetta Architecture because it inverts the original: where the historical Rosetta Stone was built for decoding (one script unlocking another), this is built for sealing — the same truth certified through six independent formal systems simultaneously.

2. Dual-Token Filter — Formal Separation of Logic and Expression

Logic is static. Voice is dynamic.

The formal substrate never changes. The expression layer is fully swappable.

The LLM synthesizes — it does not compute.

Every agent response now ingests two tokens before any LLM inference occurs:

TOKEN 1 — SYSTEM TRUTH VECTOR	TOKEN 2 — CULTURAL TEMPLATE
Deterministic output from the native language body. Mathematical. Absolute. Produced before the query reaches the LLM. The substrate has already computed the answer.	Voice codec — register, pacing, domain context. Fully swappable without touching the computation. The expression layer that the LLM reasons through.

The LLM's role is synthesis, not derivation. It cannot override the truth vector because the truth vector is not inside the LLM — it is a pre-computed fact from a formal language body that the LLM is instructed to synthesize through its voice layer.

The significance for governance: **the substrate cannot be prompted out of its invariants.** Adversarial inputs that attempt to redirect, override, or reframe agent behavior encounter a pre-committed formal result — not a probabilistic model that can be steered. The invariants live in the type system, the COBOL fixed-point arithmetic, the Prolog constraint graph. They are not negotiable at the natural language layer.

In terms of Multiplicity Theory: Token 1 is the prime — the place where structure is interrogated and a formal answer is produced. Token 2 is the expression layer — the multiplicity that survives the interrogation and reaches the surface. The thickness of what survives is determined before the LLM fires.

3 . **Live State Hydration — Self-Observing FSM**

The global state machine, as described in our prior exchange, defined transitions but did not make the system self-observing at the moment of transition. That gap is now closed. Before each state transition (agent response), the system queries its own current position and injects that reading into the truth vector. The FSM observes itself before it moves.

Four queries run in parallel, bounded by a hard timeout, before every response:

SIGNAL	SOURCE	CONTENT
Agent grade	Persistent grade ledger	Most recent performance score across five dimensions — content, domain, honesty, structure, sovereign alignment
Mesh event count	Event ledger (5-min window)	Current activity level of the full agent mesh
Kinetic layer status	Rust WORM handler	Whether the deterministic execution layer is online — 800ms hard timeout
Domain fact	Domain-specific record	Agent-specific: treasury agent reads active reserve balances; financial agent reads 24h transaction count; orchestration agent reads 1h mesh event volume

The resulting truth vector carries both the static substrate computation and the live system reading:

```
SAMPLE TRUTH VECTOR — TREASURY AGENT — INJECTED BEFORE LLM SYNTHESIS

COBOL [VAULT]: status=SEALED reserve_check=PASS veto_power=YES — Reserve threshold met. Capital
movement authorized. Dual signature pending.

| LIVE[mesh_5m=14 rust=online grade=B+(7.4) active_pools=3 total_balance=$24,500]
```

The COBOL substrate provides the invariant fact. The live hydration provides the dynamic equilibrium reading. The LLM receives both and synthesizes through its voice layer. **The agent knows where it is before it speaks.**

The architectural consequence: this is no longer a system that responds to queries — it is a system that responds to queries from a known position in a live state space. The distinction matters for governance. A governance runtime that does not observe its own state cannot make claims about its own behavior. This one can.

– **Note on Correspondence to Multiplicity Theory**

This architecture was not designed against your framework. The correspondence was identified after the substrate became executable. We flag it because the alignment is precise enough to warrant examination:

MULTIPLICITY THEORY (VAN GELDER)	THIS ARCHITECTURE
"A prime is a place where structure is interrogated"	SUBLEQ gate — every state transition is an interrogation point with a determined outcome
"Multiplicity is the thickness of what survives at that place"	WORM chain — the sealed record of every transition that passed all five passes
Executable ontology	Six language bodies making formal constraints executable — the ontology runs, it does not merely describe
Dynamic equilibrium	Live state hydration — the FSM reads its own position before each transition
PhaseMirror — AI governance runtime (Gate B)	This architecture — AI governance runtime, bare metal, in production

PhaseMirror is at Gate B. We are interested in whether the substrate architecture described here informs or challenges what you are building toward Gate C and beyond — specifically whether the SUBLEQ reduction and the dual-token separation offer any formal grounding for the Sigma Kernel layer you are working through.